Esercizio 1 (11 punti).

Sia data una CPU con processore a **8GHz** e **16 CPI** (Clock per Instruction) che adoperi indirizzi da 32 bit e memoria strutturata su due livelli di **cache** (L1, L2), il cui setup è come segue:
L1 è una cache **set-associativa** a **2 vie** con **2 set** e **blocchi da 32 word**; adopera una politica di rimpiazzo **LRU**. Ricordiamo che consideriamo la linea come l'insieme del blocco in cache con tag e bit di validità, mentre il set è il gruppo di linee con il medesimo indice.
L2 è una cache **direct-mapped** con **2 linee** e **blocchi da 256 word**.

1) Supponendo che all'inizio nessuno dei dati sia in cache, indicare quali degli accessi in memoria indicati di seguito sono HIT o MISS in ciascuna delle due cache. Per ciascuna **MISS** indicare se sia di tipo Cold Start (**Cold**), Capacità (**Cap**) o Conflitto (**Conf**). Utilizzare la tabella sottostante per fornire i risultati e indicare la metodologia di calcolo.

	Address	1000	2000	1010	4030	8020	880	2780	602	112	620	1913	770
L1	Block#												
	Index												
	Tag												
	HIT/MISS												
	Miss type												
L2	Block#												
	Index												
	Tag												
	HIT/MISS												
	Miss type												

2) Calcolare le dimensioni in bit (compresi i bit di controllo ed assumendo che ne basti uno per la LRU) delle due cache: (a) L1 e (b) L2.
3) Assumendo che gli accessi in **memoria** impieghino **400 ns**, che gli **hit** nella cache **L1** impieghino **10 ns** e gli **hit** nella cache **L2** impieghino **20 ns**, calcolare (a) il **tempo totale** per la sequenza di accessi, (b) il tempo **medio** per la sequenza di accessi, e (c) **quante istruzioni** vengono svolte nel tempo medio calcolato.
4) Calcolare il word offset del sesto indirizzo per la cache L2 spiegando i calcoli effettuati.
5) Supponendo che gli indirizzi nella tabella siano virtuali e la memoria virtuale consti di **128 pagine** di **4KiB** ciascuna, indicarne i numeri di pagina virtuale.

Address	100	2000	1010	4030	8020	880	2780	602	112	620	1913	770
Page#												

Esercizio 2 (11 punti).

Considerare l'architettura RISC-V a ciclo singolo nella figura in basso (ed in allegato).
Si vuole aggiungere l'istruzione di **tipo R** "branch if less than (safe)"
`bltsafe rd, rs1, rs2`
che prende i 5 bit di rd e li estende SENZA segno a 32 bit (chiameremo questo valore rd_{32}), e salta all'istruzione il cui indirizzo è in `PC` + rd_{32} se e solo se valgono ambedue le seguenti condizioni:
a) `rs1 < rs2`;
b) `PC` + $rd_{32} \leq$ `0x7FFFFFFF`.

Si specifica che `rs1, rs2` sono valori letti dal banco dei registri, e che si può usare l'unità Genera Cost per estendere rd a 32 bit.

1) Mostrare le **modifiche all'architettura** della CPU, avendo cura di aggiungere eventuali altri componenti necessari a realizzare l'istruzione. A tal fine, si può alterare la stampa del diagramma architetturale oppure ridisegnare i componenti interessati dalla modifica, avendo cura di indicare i fili di collegamento e tutti i segnali entranti ed uscenti. Indicare inoltre sul diagramma i **segnali di controllo** che la CU genera *per realizzare l'istruzione*.

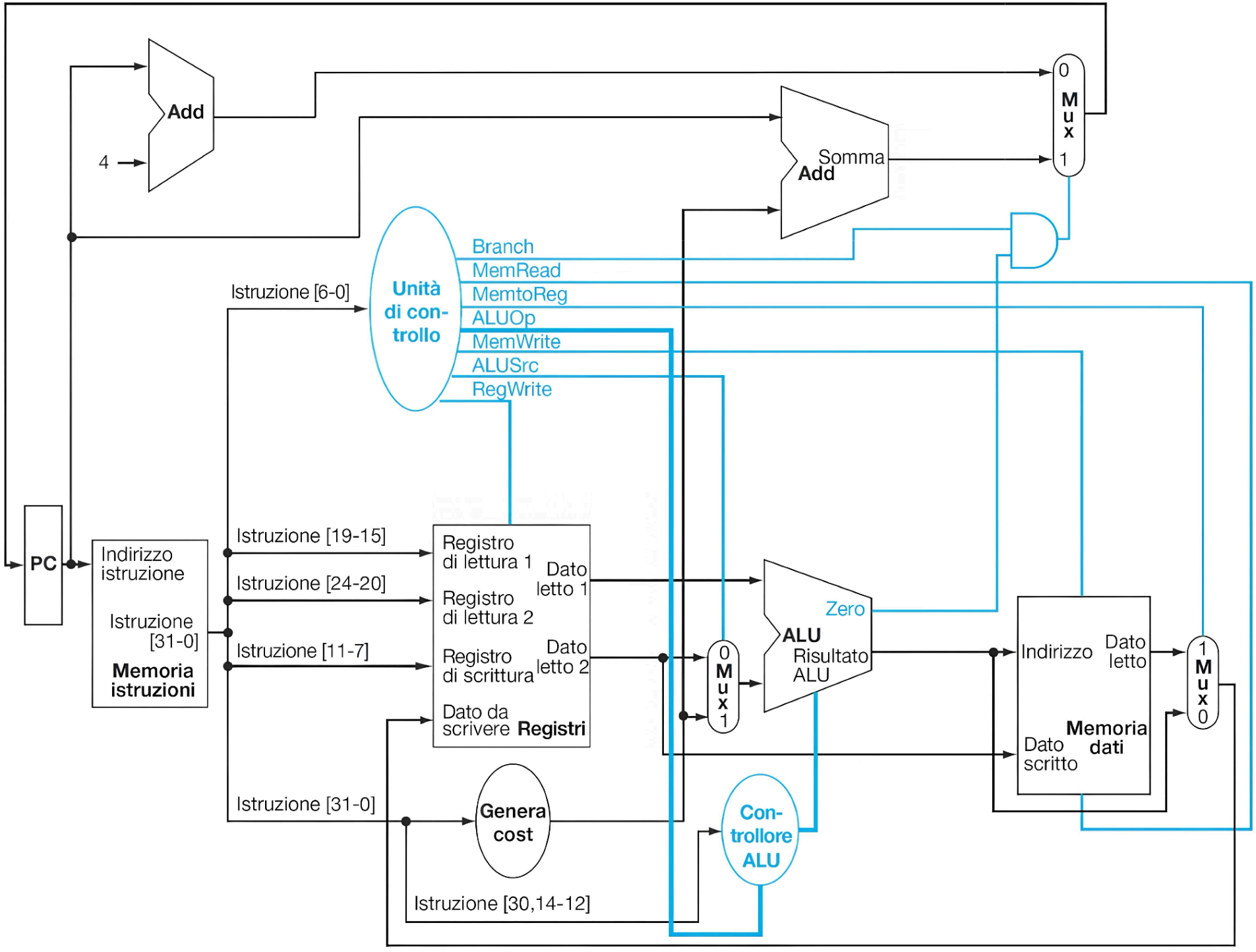
2) Indicare il contenuto in bit della word che esprime l'istruzione
`bltsafe 16, 19, 3`
compilando la tabella sottostante (assumiamo che lo *OpCode* di `bltsafe` sia `0x38`).



3) Supponendo che l'accesso alle **memorie** impieghi **200 ns**, l'accesso ai **registri 50 ns**, le operazioni dell'**ALU e** dei **sommatori 100 ns**, e che gli altri ritardi di propagazione dei segnali siano trascurabili, indicare la durata totale del **ciclo di clock** tale che anche l'esecuzione della nuova istruzione sia permessa *spiegando i calcoli effettuati*.

4) Indicando con `Blts` il segnale di controllo che viene asserito per eseguire la nuova istruzione, assumiamo che
a) tutti i segnali di tipo *don't care* siano pari a `0` e che
b) la Control Unit della CPU modificata per supportare `bltsafe` sia difettosa e sovrascriva il segnale `RegWrite` come segue:
`RegWrite` ≔ `Blts`
In tal caso, quale sarà il valore di `s1` al termine dell'esecuzione del seguente frammento di codice assumendo che `s0` e `s1` siano inizializzati a `0`? Spiegare il motivo.

```
addi    s0, zero, 2
addi    s1, zero, 1
bltsafe s0, s1, 5
```


Add
4
Add
Somma
0
M
u
x
1
Branch
MemRead
MemtoReg
ALUOp
MemWrite
ALUSrc
RegWrite
Unità di con-trollo
Istruzione [6-0]
PC
Indirizzo istruzione
Istruzione [31-0]
Memoria istruzioni
Istruzione [19-15]
Istruzione [24-20]
Istruzione [11-7]
Registro di lettura 1
Registro di lettura 2
Registro di scrittura
Dato da scrivere
Dato letto 1
Dato letto 2
Registri
Zero
ALU
Risultato ALU
Indirizzo
Dato letto
Dato scritto
Memoria dati
Istruzione [31-0]
Genera cost
Con-trollore ALU
Istruzione [30,14-12]

Esercizio 3 (11 punti)

Si consideri l'architettura RISC-V con pipeline. Il programma qui di seguito effettua la conversione delle sequenze di numeri binari separati da “/” in ottale considerando solo le triplette intere (cioè eventuali coppie o numeri singoli verranno scartati) e stampa questi numeri su righe successive.

```
1  .data
2      buffer: .asciz "111/101000110/1111"
3  .text
4  la a0, buffer
5  addi s0, zero, 47               # separatore "/"
6  addi s1, zero, 0                # carattere nullo
7  mv s2, a0                       # indirizzo stringa
8  gruppoTre:
9     addi t0, zero, 3             # di quanto shiftare
10    addi t2, zero, 0             # accumulatore
11 loopGruppo:
12    addi t0, t0, -1              # contatore gruppo decrementa
13    lb t1, 0(s2)                 # carica carattere
14    addi s2, s2, 1               # prossimo
15    beq  t1, zero, exit          # esci se fine stringa
16    beq  t1, s0, fineGruppo      # se separatore, stampa a capo e ricomincia
17    addi t1, t1, -48             # trasformalo in intero
18    sll t1, t1, t0               # spostalo
19    add t2, t2, t1               # accumula
20    beq  t0, zero, fineGTre      # raggiunti tre, stampa numero e ricomincia gruppotre
21    j loopGruppo
22 fineGTre:
23    mv a0, t2                    # Carica il risultato per la stampa
24    addi a7, zero, 1             # Imposta stampa intero
25    ecall
26    j gruppoTre
27 fineGruppo:
28    addi a0, zero, 10            # newline
29    addi a7, zero, 11              Imposta stampa carattere
30    ecall                        # Stampa newline
31    j gruppoTre
32 exit:
```

Si supponga che non si faccia uso di alcuna pseudoistruzione, e che le ecall non richiedano alcuno stallo. Si consideri che la decisione di branch venga presa in fase di ID, e che le unità di forwarding (quando usate) includano il forwarding da EXE a EXE, da MEM a EXE, e (per il branch) da EXE a ID.
Si indichino (ignorando hazard che possano concernere la ecall):

1) Per i primi 24 cicli di clock (**senza unità di forwarding**), le istruzioni tra le quali sono presenti **data hazard** – indicare i numeri di linea N ed M (visibili alla sinistra delle linee di codice in figura) ed il registro coinvolto xY (nel formato N / M / xY, ad esempio 17 / 18 / t0 se c'è un data hazard causato dal registro t0 tra l'istruzione alla linea 17 e quella alla linea 18);

2) Per i primi 24 cicli di clock (**con unità di forwarding**) le istruzioni tra le quali sono presenti **control hazard** – indicare i numeri di linea N ed M (nel formato N / M, ad esempio se l'istruzione alla linea 17 causa un control hazard con l'istruzione alla linea 23 scrivere: 17 / 23);

3) quanti **cicli di clock** sono necessari ad eseguire il programma tramite **forwarding**, spiegando il calcolo effettuato;

4) quanti **cicli di clock** sarebbero necessari ad eseguire il programma **senza forwarding**, spiegando il calcolo effettuato;

5) quali sono, per ognuna delle cinque fasi, le **istruzioni** (o le bolle) in pipeline durante il **16° ciclo di clock** (**con forwarding**);
   IF:
   ID:
   EX:
   MEM:
   WB:

6) **Domanda teorica sulla pipeline qui**